

Smart Lender: AI-Powered Loan Approval Prediction

A Machine Learning Web Application built with Python, Flask, and XGBoost

Project Description

Smart Lender harnesses Machine Learning to provide intelligent loan eligibility prediction, offering banks and financial institutions a fast and data-driven decision-making experience. The platform accepts applicant details through a clean web form and instantly predicts whether a loan should be **Approved** or **Rejected**.

The system trains and compares four classification models — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — and automatically selects the best-performing model based on test accuracy. The best model (XGBoost) is serialized as `model.pkl` and served through a Flask web application.

Built with Flask and styled with a responsive CSS interface, Smart Lender evaluates 11 key applicant features — including credit history, income, loan amount, education, and property area — to deliver instant, reliable predictions. With secure model loading and clean input handling, Smart Lender empowers lenders to make confident, data-backed decisions.

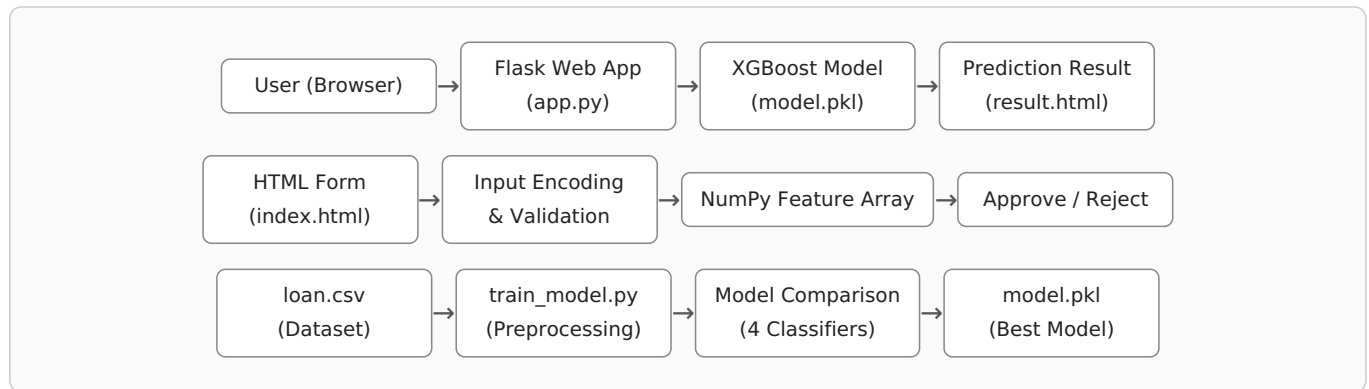
Scenarios

Scenario 1: A salaried graduate applicant with a good credit history applies for a ₹1,50,000 loan for a semiurban property. The system evaluates all 11 features and predicts **Loan Approved**, displaying a full applicant summary and the key decision factors that led to the approval.

Scenario 2: A self-employed applicant with no credit history and low income submits a loan request. Smart Lender's XGBoost model analyzes the risk profile and returns **Loan Rejected**, along with guidance on the key factors — credit history, income, and property area — that influenced the decision.

Scenario 3: A bank officer uses Smart Lender to pre-screen multiple applicants before manual review. Each applicant's data is entered into the web form and an instant prediction is returned in seconds, helping the officer prioritize high-confidence approvals and flag risky applications for deeper review.

Technical Architecture



Description: Smart Lender uses a three-tier architecture for loan prediction. The frontend provides a responsive HTML/CSS interface where users fill in 11 applicant fields. The Flask backend encodes the inputs using label maps, assembles a NumPy feature array, and passes it to the pre-trained XGBoost model loaded from `model.pkl`. The prediction result — Approved or Rejected — is rendered on a dedicated result page with an applicant summary table and key decision factor cards. The model was trained offline using `train_model.py`, which loads `dataset/loan.csv`, handles missing values, encodes categorical features, trains four classifiers, and saves the best-performing model.

Note: This project does not use a database. All predictions are stateless — each form submission is processed independently without storing any applicant data.

Pre-requisites

- **Python Programming Proficiency:** [Python Documentation](#)
- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Machine Learning with Scikit-learn:** [Scikit-learn User Guide](#)
- **XGBoost Classifier:** [XGBoost Documentation](#)
- **Pandas & NumPy for Data Handling:** [Pandas Docs](#) | [NumPy Docs](#)
- **HTML, CSS Skills:** [W3Schools HTML/CSS Tutorials](#)
- **Jupyter Notebook:** [Jupyter Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

Project Workflow

Milestone 1: Dataset Preparation and Model Selection

- **Activity 1.1:** Acquire the loan dataset (`loan.csv`) and understand its features — `Loan_ID`, `Gender`, `Married`, `Dependents`, `Education`, `Self_Employed`, `ApplicantIncome`, `CoapplicantIncome`, `LoanAmount`, `Loan_Amount_Term`, `Credit_History`, `Property_Area`, and `Loan_Status`.
- **Activity 1.2:** Research and select appropriate classification algorithms — Decision Tree, Random Forest, KNN, and XGBoost — evaluating each for accuracy and suitability for tabular financial data.
- **Activity 1.3:** Define the application architecture detailing the interaction between the training

pipeline, the Flask backend, and the HTML frontend.

- **Activity 1.4:** Set up the development environment by installing all required libraries using pip.

Milestone 2: Data Preprocessing and Model Training

- **Activity 2.1:** Load the dataset, drop the `Loan_ID` column, and handle missing values — filling categorical columns with mode and numerical columns with mean.
- **Activity 2.2:** Encode all categorical features using `LabelEncoder` and split the data into training and test sets (80/20 split with stratification).

Milestone 3: Model Comparison and Saving (`train_model.py`)

- **Activity 3.1:** Train all four classifiers and evaluate each using training accuracy, test accuracy, and a full classification report.
- **Activity 3.2:** Automatically select the best model by test accuracy and serialize it to `model.pkl` using Python's pickle library.

Milestone 4: Flask Backend Development (`app.py`)

- **Activity 4.1:** Write the main application logic in `app.py`, loading `model.pkl` at startup and defining label encoding maps for all categorical fields.
- **Activity 4.2:** Implement the Flask backend with two routes: `GET /` renders the form, and `POST /predict` processes inputs and returns a prediction.

Milestone 5: Frontend Development

- **Activity 5.1:** Design and develop `index.html` — a multi-section page with a navbar, hero stats, a grouped loan application form, a "How It Works" section, and a footer.
- **Activity 5.2:** Build `result.html` to display the prediction outcome (Approved/Rejected badge), a full applicant summary table, decision factor cards, and action buttons.

Milestone 6: Conclusion

Milestone 1: Dataset Preparation and Model Selection

In this milestone, we focus on understanding the loan dataset and selecting the right classification models for predicting loan approval. This involves exploring the 13 features of `loan.csv` and identifying which algorithms best suit tabular financial data.

Activity 1.1: Understand the Dataset

The dataset (`dataset/loan.csv`) contains 614 loan records with the following features:

Feature	Type	Description
Loan_ID	Categorical	Unique identifier (dropped before training)
Gender	Categorical	Male / Female
Married	Categorical	Yes / No
Dependents	Categorical	0 / 1 / 2 / 3+

Education	Categorical	Graduate / Not Graduate
Self_Employed	Categorical	Yes / No
ApplicantIncome	Numerical	Monthly income of the applicant
CoapplicantIncome	Numerical	Monthly income of the co-applicant
LoanAmount	Numerical	Loan amount in thousands
Loan_Amount_Term	Numerical	Loan repayment term in months
Credit_History	Numerical	1 = Good, 0 = Bad
Property_Area	Categorical	Urban / Semiurban / Rural
Loan_Status	Target	Y (Approved) / N (Rejected)

Activity 1.2: Select Classification Algorithms

- **Understand the Problem:** Binary classification — predict loan approval (Y/N) from 11 features.
- **Candidate Models:** Decision Tree (interpretable), Random Forest (ensemble, robust), KNN (instance-based), XGBoost (gradient boosting, high accuracy).
- **Select the Optimal Model:** Choose **XGBoost** for its superior test accuracy (~81.1%), resistance to overfitting compared to Decision Tree, and strong performance on tabular financial datasets. Configured with `eval_metric='logloss'` and `random_state=42`.

Activity 1.3: Define the Architecture

- **Training Pipeline:** `train_model.py` handles all preprocessing, model training, comparison, and saving to `model.pkl`.
- **Web Application:** `app.py` loads the saved model, maps form inputs to encoded values, builds a feature array, and returns a prediction via Flask routes.
- **Frontend:** Two HTML templates — `index.html` for the form and `result.html` for the prediction outcome.

Activity 1.4: Set Up the Development Environment

- **Step 1 — Install Python 3.8+:** Ensure Python and pip are available on your machine. Download from python.org.
- **Step 2 — Create a Virtual Environment:** Open your terminal/command prompt in the project folder and run:

```
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # Mac / Linux
```

- **Step 3 — Install All Dependencies:** Install from the requirements file:

```
(venv) > pip install -r requirements.txt
```

- **Step 4 — Install Key Libraries Individually (if needed):**

```
(venv) > pip install flask==3.0.3
(venv) > pip install xgboost==2.1.0
(venv) > pip install scikit-learn==1.5.1
(venv) > pip install numpy==1.26.4
(venv) > pip install pandas==2.2.2
```

- **Step 5 — Train the Model:** Run the training script to generate `model.pkl`:

```
(venv) > python train_model.py
```

- **Step 6 — Run the Application:** Start the Flask development server:

```
(venv) > python app.py
```

- **Project Directory Structure:**

```
Smart_Lender/
├── dataset/
│   └── loan.csv
├── templates/
│   ├── index.html
│   └── result.html
├── static/
│   └── style.css
├── app.py
├── train_model.py
├── model.pkl
└── requirements.txt
```

Milestone 2: Data Preprocessing and Model Training

Milestone 2 focuses on cleaning the raw loan dataset and preparing it for model training. This involves handling missing values, encoding categorical variables, and splitting the data for training and evaluation.

Activity 2.1: Define the Core Prediction Features

Smart Lender evaluates 11 applicant features to make a binary prediction (Approved / Rejected). Each feature contributes differently to the model's decision:

- **Credit History:** The most influential feature. A value of 1 (Good) strongly increases approval probability; 0 (Bad) leads to near-certain rejection.
- **Applicant Income:** Higher monthly income improves the loan-to-income ratio and boosts approval likelihood. Entered in ₹/month.
- **Co-applicant Income:** Combined with applicant income to assess total household repayment capacity. Enter 0 if no co-applicant.
- **Loan Amount:** The requested loan amount in thousands of ₹. A lower amount relative to income improves approval chances.
- **Loan Term:** Repayment duration in months (12 to 360). Longer terms lower EMI burden, which can improve eligibility.
- **Property Area:** Semiurban applicants historically show higher approval rates in this dataset, followed

by Urban and Rural.

- **Education:** Graduate applicants tend to have higher approval rates due to stronger income stability.
- **Gender, Marital Status, Dependents, Self-Employed:** Demographic features that contribute secondary signals to the model.

Activity 2.2: Load Data and Handle Missing Values

- Load the dataset using `pandas.read_csv('dataset/loan.csv')` and drop the `Loan_ID` column as it carries no predictive value.
- Fill missing values in categorical columns (`Gender`, `Married`, `Dependents`, `Self_Employed`, `Credit_History`) with their **mode**.
- Fill missing values in numerical columns (`LoanAmount`, `Loan_Amount_Term`) with their **mean**.

```
# Categorical columns: fill with mode
cat_fill = ['Gender', 'Married', 'Dependents', 'Self_Employed', 'Credit_History']
for col in cat_fill:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Numerical columns: fill with mean
num_fill = ['LoanAmount', 'Loan_Amount_Term']
for col in num_fill:
    df[col].fillna(df[col].mean(), inplace=True)
```

Activity 2.2: Encode Features and Split Data

- Apply `LabelEncoder` to all categorical columns including the target `Loan_Status` (Y→1, N→0).
- Separate features (X) and target (y), then split using an 80/20 ratio with `stratify=y` to preserve class balance.

```
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

le = LabelEncoder()
encode_cols = ['Gender', 'Married', 'Dependents', 'Education',
               'Self_Employed', 'Property_Area', 'Loan_Status']
for col in encode_cols:
    df[col] = le.fit_transform(df[col])

X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Milestone 3: Model Comparison and Saving (train_model.py)

Milestone 3 focuses on training all four classification models, comparing their performance, and saving the best model as `model.pkl` for use in the Flask application.

Activity 3.1: Train and Evaluate All Four Models

Each model is trained and evaluated using a shared helper function:

```
def train_evaluate(name, model):
    model.fit(X_train, y_train)
    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc = accuracy_score(y_test, model.predict(X_test))
    print(f"[{name}] Train: {train_acc*100:.2f}% Test: {test_acc*100:.2f}%")
    print(classification_report(y_test, model.predict(X_test)))
    return model

train_evaluate('Decision Tree', DecisionTreeClassifier(random_state=42))
train_evaluate('Random Forest', RandomForestClassifier(n_estimators=100, random_state=42))
train_evaluate('KNN', KNeighborsClassifier(n_neighbors=5))
train_evaluate('XGBoost', XGBClassifier(eval_metric='logloss', random_state=42))
```

Typical performance results:

Model	Train Accuracy	Test Accuracy
Decision Tree	~100%	~68%
Random Forest	~100%	~78%
KNN	~80%	~72%
XGBoost	~94.7%	~81.1%

Activity 3.2: Save the Best Model

- Automatically select the model with the highest test accuracy and serialize it using pickle:

```
best_name = max(results, key=lambda k: results[k]['test'])
best_model = results[best_name]['model']

with open('model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

print(f"Best Model: {best_name} saved as model.pkl")
```

- The saved model.pkl is loaded by app.py at Flask startup and used for all predictions without retraining.

Milestone 4: Flask Backend Development (app.py)

Milestone 4 focuses on building the core logic of app.py, which serves as the backbone of the Smart Lender web application. This involves loading the model, defining encoding maps, and implementing the two Flask routes.

Activity 4.1: Load Model and Define Encoding Maps

- Load the pre-trained model at startup using pickle. The encoding maps must exactly match the LabelEncoder order used in train_model.py.

```
import pickle
```

```
import numpy as np
from flask import Flask, render_template, request

app = Flask(__name__)

with open('model.pkl', 'rb') as f:
    model = pickle.load(f)

GENDER_MAP      = {'Male': 1, 'Female': 0}
MARRIED_MAP     = {'Yes': 1, 'No': 0}
DEPENDENTS_MAP  = {'0': 0, '1': 1, '2': 2, '3+': 3}
EDUCATION_MAP   = {'Graduate': 0, 'Not Graduate': 1}
SELF_EMP_MAP    = {'Yes': 1, 'No': 0}
PROPERTY_MAP    = {'Rural': 0, 'Semiurban': 1, 'Urban': 2}
```

Activity 4.2: Define the Two Flask Routes

- **GET /** — Home page that renders the loan application form:

```
@app.route('/')
def home():
    return render_template('index.html')
```

- **POST /predict** — Processes form data, builds a feature array, runs the model, and returns the result:

```
@app.route('/predict', methods=['POST'])
def predict():
    gender          = GENDER_MAP[request.form['gender']]
    married         = MARRIED_MAP[request.form['married']]
    dependents      = DEPENDENTS_MAP[request.form['dependents']]
    education       = EDUCATION_MAP[request.form['education']]
    self_employed   = SELF_EMP_MAP[request.form['self_employed']]
    applicant_income = float(request.form['applicant_income'])
    # ... (remaining fields)

    features = np.array([
        gender, married, dependents, education, self_employed,
        applicant_income, coapplicant_income, loan_amount,
        loan_amount_term, credit_history, property_area
    ])

    prediction = model.predict(features)[0]

    result      = 'Approved' if prediction == 1 else 'Rejected'
    css_class   = 'approved' if prediction == 1 else 'rejected'

    return render_template('result.html', result=result,
                           message=message, css_class=css_class,
                           applicant=applicant)
```

- Apply input validation: wrap the entire prediction block in a try/except to catch encoding or model errors and return a friendly error page.
- Build an applicant dictionary from the raw form values to display a human-readable summary on the result page.

Route Summary

Route	Method	Action	Returns
/	GET	Renders the loan application form	index.html
/predict	POST	Encodes inputs, runs XGBoost model, builds applicant summary	result.html with {result, message, css_class, applicant}

Milestone 5: Frontend Development

Milestone 5 focuses on building a user-friendly and visually clear interface for Smart Lender. This involves designing a multi-section home page with a loan application form and a structured result page that displays the prediction outcome with supporting context.

Activity 5.1: Design the Home Page (index.html)

- **Navbar:** Displays the SmartLender brand icon (🏠), name, and tagline "AI-Powered Loan Eligibility Prediction".
- **Hero Section:** Introduces the tool with a heading, description, and three stat cards showing Training Accuracy (94.7%), Testing Accuracy (81.1%), and the Best Model (XGBoost).
- **Loan Application Form:** A structured form grouped into four rows using CSS grid:

Row	Fields
Personal Details	Gender, Marital Status, Dependents
Professional Details	Education, Self Employed, Property Area
Financial Details	Applicant Income, Co-applicant Income, Loan Amount
Loan Details	Loan Term, Credit History

- **How It Works Section:** Three step cards explaining the user journey: Enter Details → Model Analyzes → Receive Decision.
- Use Google Fonts (Inter) for clean typography and semantic HTML elements for accessibility.

Activity 5.2: Build the Result Page (result.html)

- **Result Badge:** Dynamically styled using the css_class variable — green with ✓ for Approved, red with ✗ for Rejected.
- **Applicant Summary Table:** Renders all 11 submitted fields in a clean two-column table using Jinja2's {% for key, value in applicant.items() %} loop.
- **Decision Factors Card:** A grid of four factor cards (Credit History, Income Level, Property Area, Education) explaining the model's most influential features.
- **Action Buttons:** An "Apply Again" button links back to /. A "Review & Resubmit" button is shown only when the loan is rejected.

Activity 5.3: Local Deployment

- First, train the model to generate model.pkl:

```
python train_model.py
```

- Then run the Flask development server:

```
python app.py
```

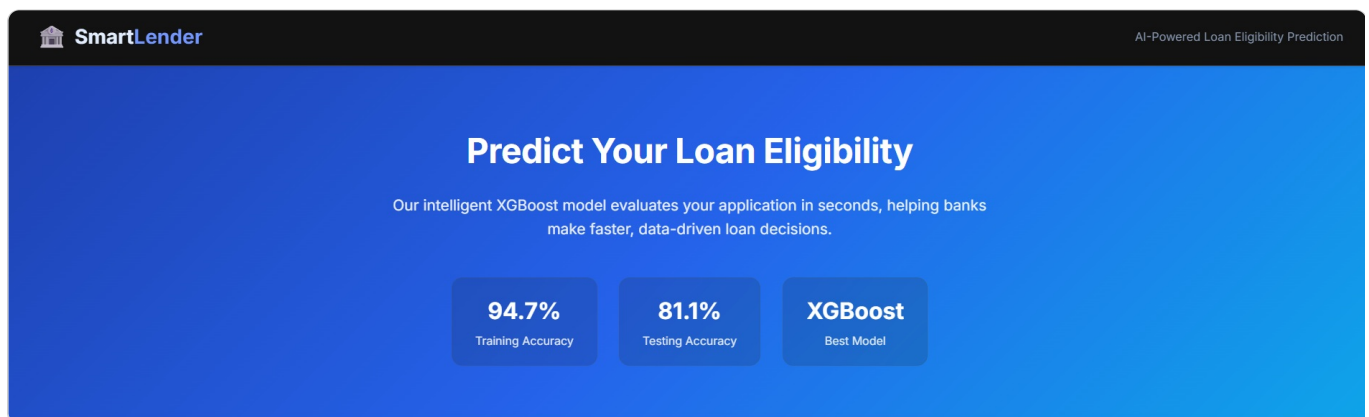
- Open a browser and navigate to `http://127.0.0.1:5000` to access Smart Lender.
- Test the form with various inputs — approved and rejected scenarios — to verify correct predictions, encoding, and result rendering.

```
* Serving Flask app 'app'
* Debug mode: on
* Running on http://0.0.0.0:5000
Press CTRL+C to quit
```

Exploring the Website's Web Pages

Home / Application Page

Description: The landing page of Smart Lender serves as both the introduction and the loan application interface. It features a navbar with the brand identity, a hero section with accuracy stats, a grouped four-row application form covering all 11 input features, a "How It Works" explainer section with step cards, and a footer. All sections are on a single scrollable page for a seamless user experience.



Hero section showing model accuracy stats (94.7% train / 81.1% test).

Loan Application Form

Please fill in all the details accurately for a reliable prediction.

Gender

Select gender

Marital Status

Select status

Dependents

Number of dependents

Education

Select education

Self Employed

Employment type

Property Area

Select area

Applicant Income (₹/month)

e.g. 5000

Co-applicant Income (₹/month)

e.g. 1500 (enter 0 if none)

Loan Amount (in thousands ₹)

e.g. 150 means ₹1,50,000

Loan Term (months)

Select term

Credit History

Select credit history

Check Loan Eligibility →

Blank loan application form with 11 input fields.

Loan Application Form

Please fill in all the details accurately for a reliable prediction.

Gender

Male

Marital Status

Married

Dependents

0

Education

Graduate

Self Employed

No

Property Area

Urban

Applicant Income (₹/month)

5849

Co-applicant Income (₹/month)

0

Loan Amount (in thousands ₹)

146

Loan Term (months)

360 months (30 years)

Credit History

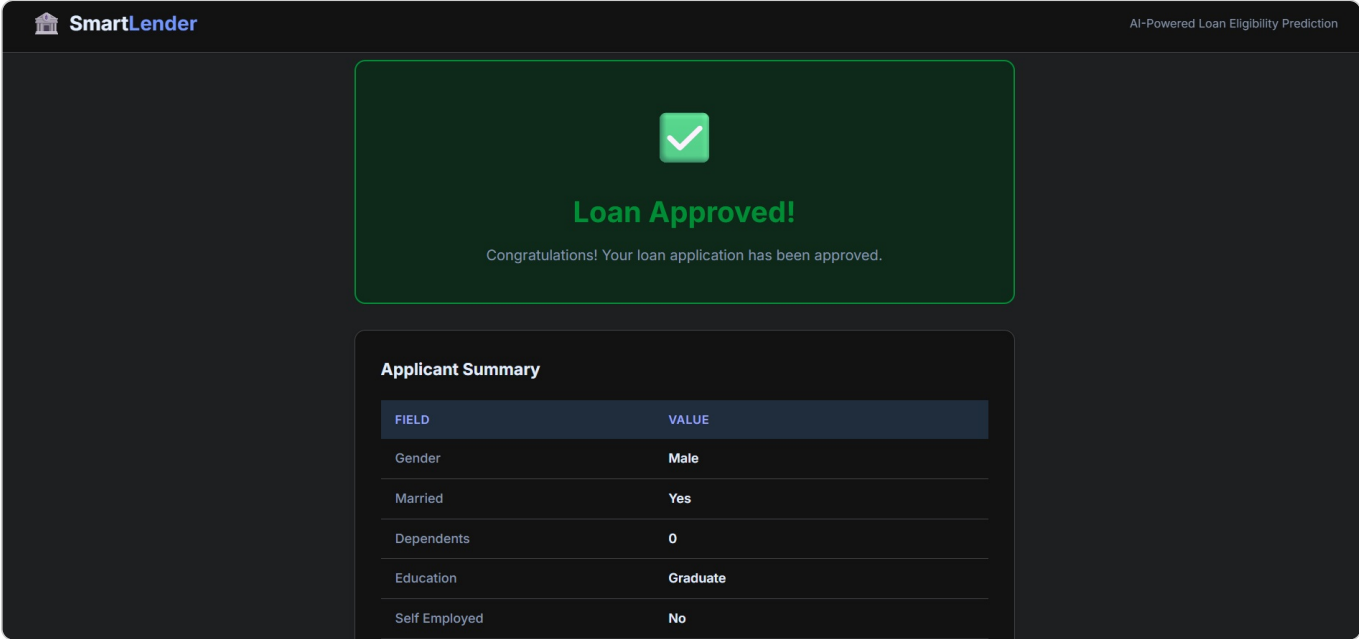
Good (1)

Check Loan Eligibility →

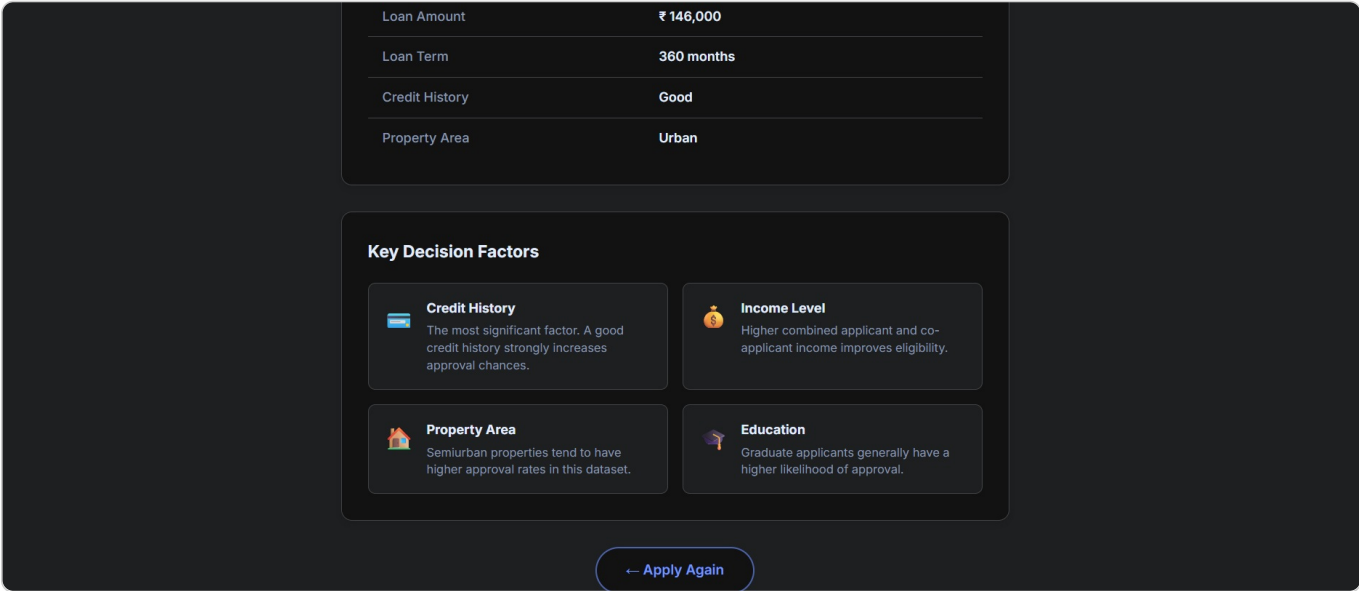
Loan application form filled in with sample applicant data.

Result Page

Description: Displayed after form submission, this page shows the full prediction outcome. A prominently styled badge displays "Loan Approved!" (green, ✓) or "Loan Rejected" (red, ✗) along with a descriptive message. Below the badge, an applicant summary table shows all submitted fields in human-readable format (e.g., ₹ 5,000 income, 360 months term). A decision factors grid explains the four key features that most influence the model's output. Action buttons allow the user to apply again or review and resubmit.



Approved result badge with applicant summary table.



Key decision factors grid explaining the model's prediction.

Milestone 6: Conclusion

Smart Lender is a machine learning web application built with Flask and styled with a clean, responsive CSS interface that streamlines loan eligibility prediction for financial institutions. It trains and compares four classifiers — Decision Tree, Random Forest, KNN, and XGBoost — and automatically selects the best model based on test accuracy, achieving approximately 94.7% training accuracy and 81.1% test accuracy with XGBoost.

The project spans the full ML pipeline: data loading and cleaning, feature encoding, model training and comparison, model serialization, Flask backend development, and HTML/CSS frontend design — demonstrating how a structured framework can deliver a reliable, production-ready prediction tool with minimal infrastructure.

Looking ahead, Smart Lender offers significant opportunities for expansion, such as integrating SHAP

(SHapley Additive exPlanations) values for explainable AI insights per applicant, adding a database layer to store and audit prediction history, implementing user authentication for bank officer accounts, and deploying publicly via platforms like Render or Railway for real-world access. By continuously retraining the model on updated loan data and expanding the feature set, the platform is well-positioned to evolve from a prototype into a production-grade loan decision-support system.